
Project 4: Website Development using AWS

Internship: Cloud AI with AWS **Organization:** *Cloudcredits* Technologies Pvt. Ltd.

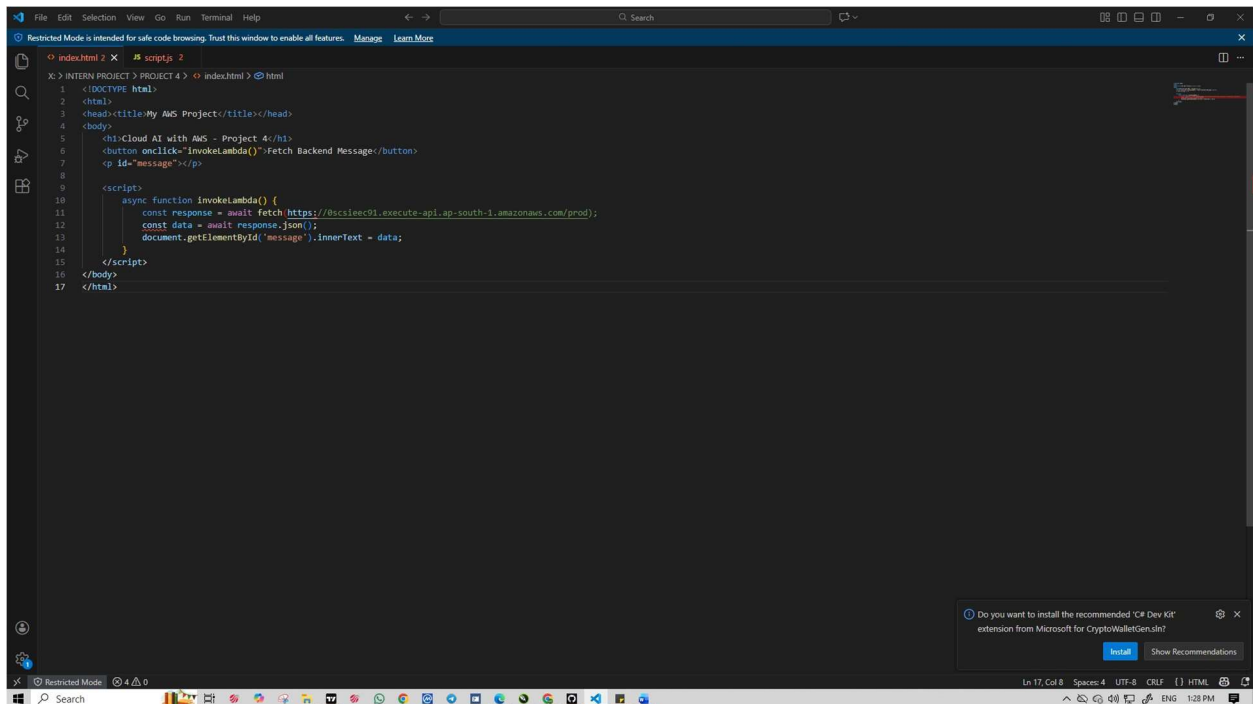
+1

Phase 1: Local Development

Step 1: Create a project folder on your computer.

Step 2: Create an index.html file. This file will serve as the frontend of your website.

Step 3: Write a simple script inside your HTML to call an API.



The screenshot shows a code editor window with a dark theme. The file explorer on the left shows a project named 'PROJECT 4' containing an 'index.html' file. The editor displays the following code:

```
1 <!DOCTYPE html>
2 <html>
3 <head><title>My AWS Project</title></head>
4 <body>
5 <h1>Cloud AI with AWS - Project 4</h1>
6 <button onclick="invokeLambda()">Fetch Backend Message</button>
7 <p id="message"></p>
8
9 <script>
10   async function invokeLambda() {
11     const response = await fetch('https://scslec91.execute-api.ap-south-1.amazonaws.com/prod');
12     const data = await response.json();
13     document.getElementById('message').innerText = data;
14   }
15 </script>
16 </body>
17 </html>
```

A notification at the bottom right asks: "Do you want to install the recommended 'C# Dev Kit' extension from Microsoft for Visual Studio Code?" with buttons for "Install" and "Show Recommendations". The status bar at the bottom indicates "Ln 17, Col 8", "Spaces: 4", "UTF-8", "CRLF", and "HTML".

File Edit Selection View Go Run Terminal Help

Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

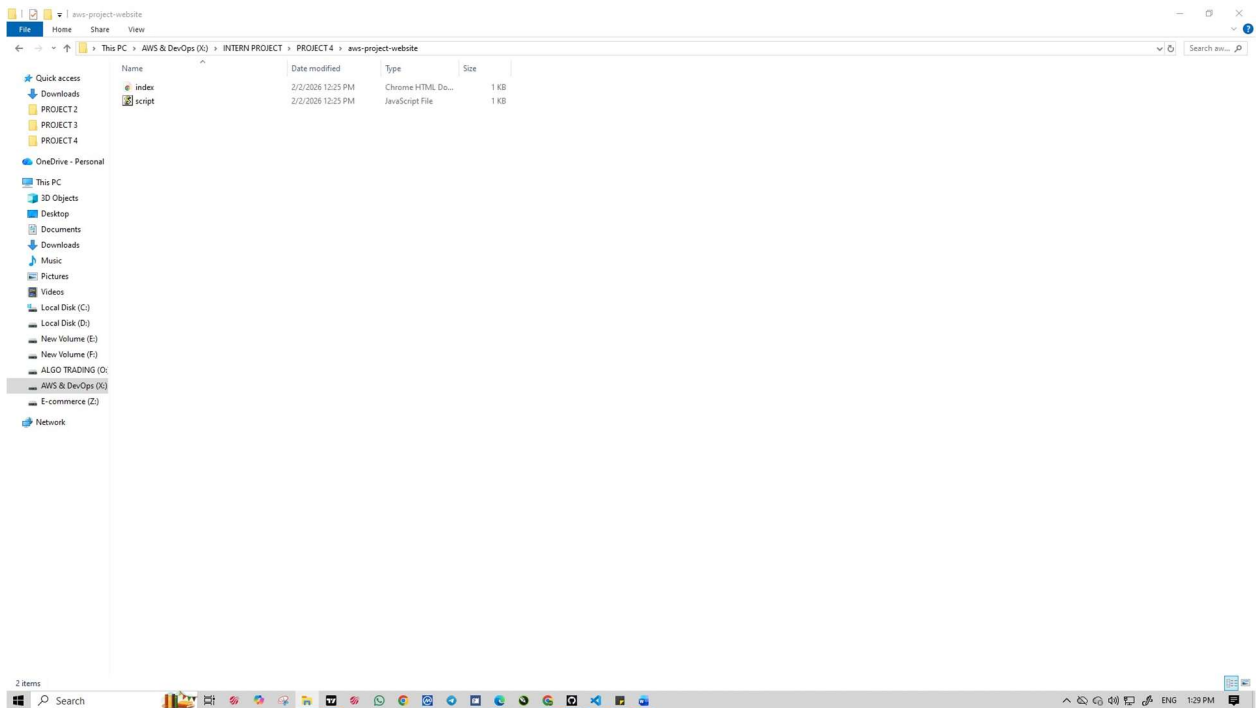
index.html 2 script.js 2 X

X: > INTERN PROJECT > PROJECT 4 > # script.js > ...

```
1
2
3   "Version": "2012-10-17",
4   "Statement": [{
5     "Sid": "PublicReadGetObject",
6     "Effect": "Allow",
7     "Principal": "*",
8     "Action": "s3:GetObject",
9     "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*"
10  ]
11
```

Ln 10, Col 2 Spaces: 4 UTF-8 CRLF JavaScript

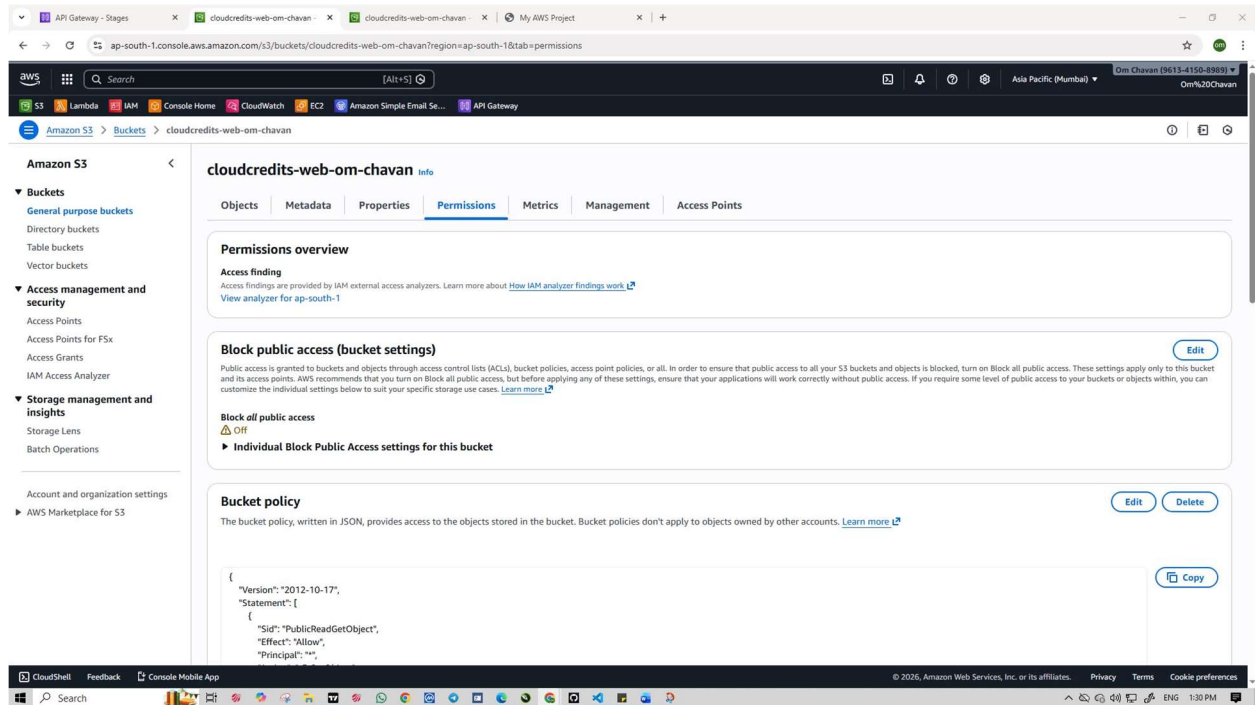
Do you want to install the recommended C# Dev Kit extension from Microsoft for CryptoWalletGen.dll? Install Show Recommendations



Phase 2: Storage & Hosting (Amazon S3)

Step 1: Create an S3 bucket with a unique name. **Step 2:** Go to **Permissions** and disable "Block all public access". **Step 3:** Scroll to the bottom of the **Properties** tab and enable **Static Website Hosting**. **Step 4:** Add a **Bucket Policy** to allow public read access to your files.

+3



Phase 3: Serverless Backend (AWS Lambda)

Step 1: Create a new Lambda function. **Step 2:** Write a simple Python or Node.js function that returns a "Hello World" or "Success" message. **Step 3:** Click **Deploy**.

The screenshot displays the AWS Lambda console for a function named 'WebsiteBackendHandler'. The 'Function overview' section shows the function's name, layers, and a description. The 'Code source' section is active, showing a Python function that returns a 'Hello! This message is coming from an AWS Lambda function!' message. The function is configured with a 'HelloWorld' trigger and a 'HelloWorld' destination. The 'Deploy' button is visible in the bottom left corner of the code editor.

Function overview

- Function name: WebsiteBackendHandler
- Layers: (0)
- Description: -
- Last modified: 28 minutes ago
- Function ARN: arn:aws:lambda:ap-south-1:961341508989:function:WebsiteBackendHandler
- Function URL: info

Code source

```
1 import json
2
3 def lambda_handler(event, context):
4     message = "Hello! This message is coming from an AWS Lambda function!"
5     return {
6         'statusCode': 200,
7         'headers': {
8             'Access-Control-Allow-Origin': '*', # Required for CORS
9             'Content-Type': 'application/json'
10        },
11        'body': json.dumps(message)
12    }
```

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

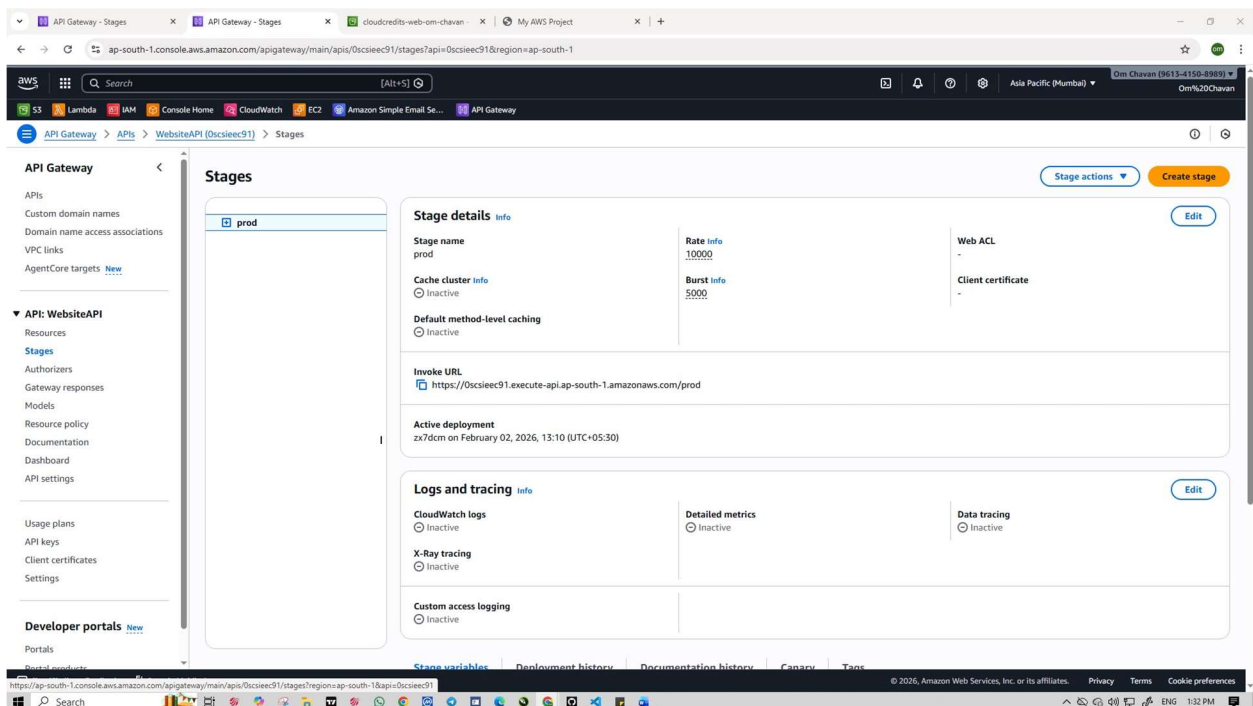
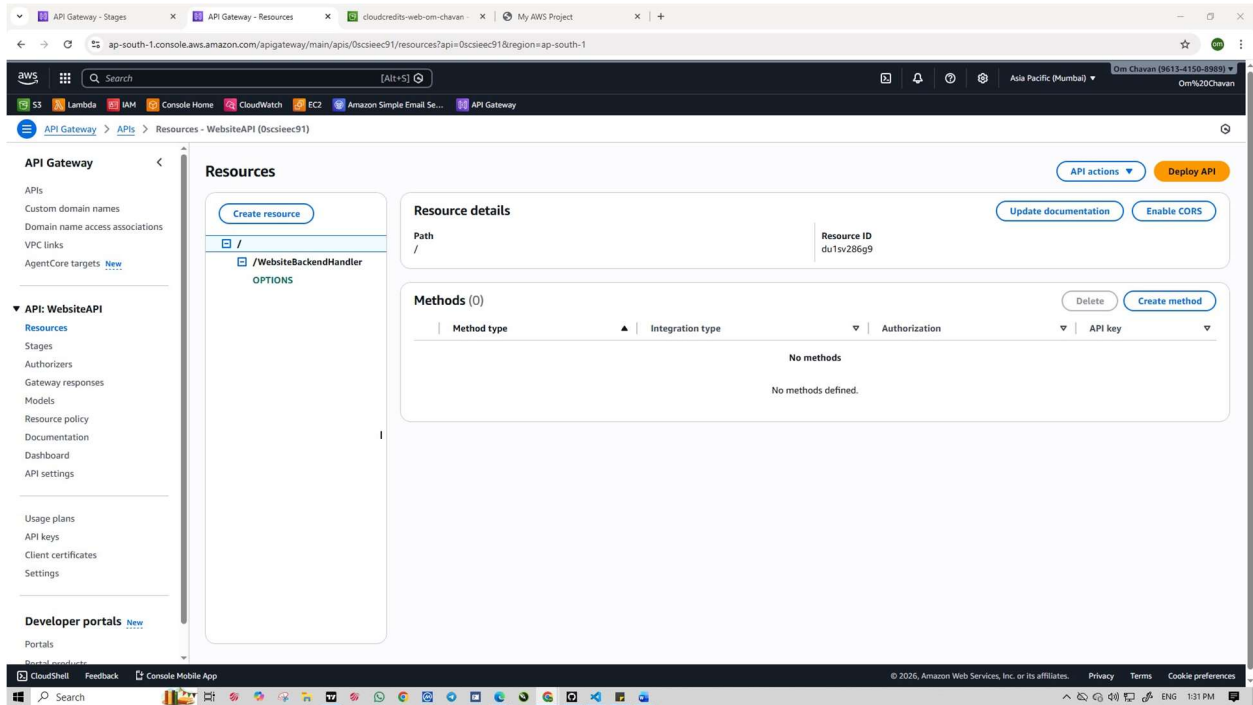
TEST EVENTS (NONE SELECTED)

ENVIRONMENT VARIABLES

Lambda Deployed @ 0.0.0 Amazon Q

Phase 4: Connecting the Bridge (API Gateway)

Step 1: Create a **REST API** in Amazon API Gateway. **Step 2:** Create a **GET** method and link it to your Lambda function. **Step 3:** Enable **CORS** (Cross-Origin Resource Sharing) so your S3 site can talk to the API. **Step 4:** **Deploy** the API and copy the **Invoke URL**.



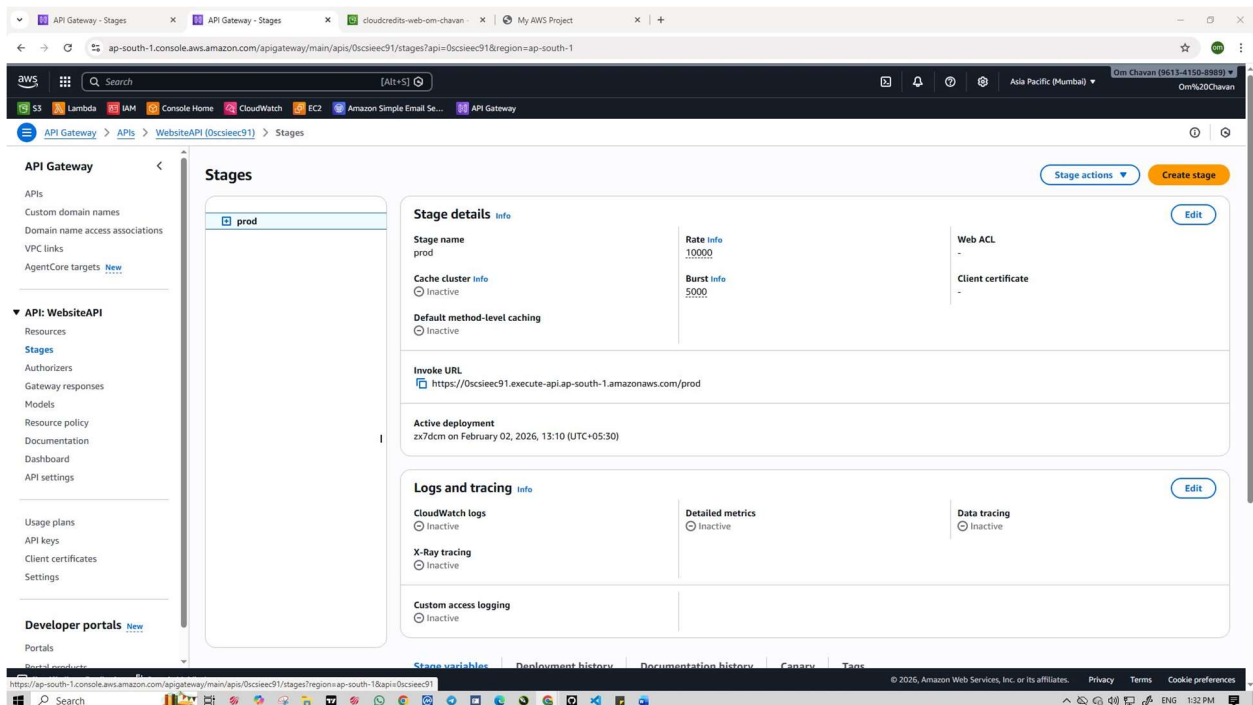
Phase 5: Testing & Final Integration

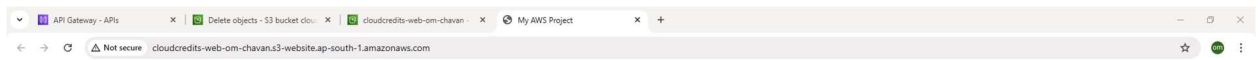
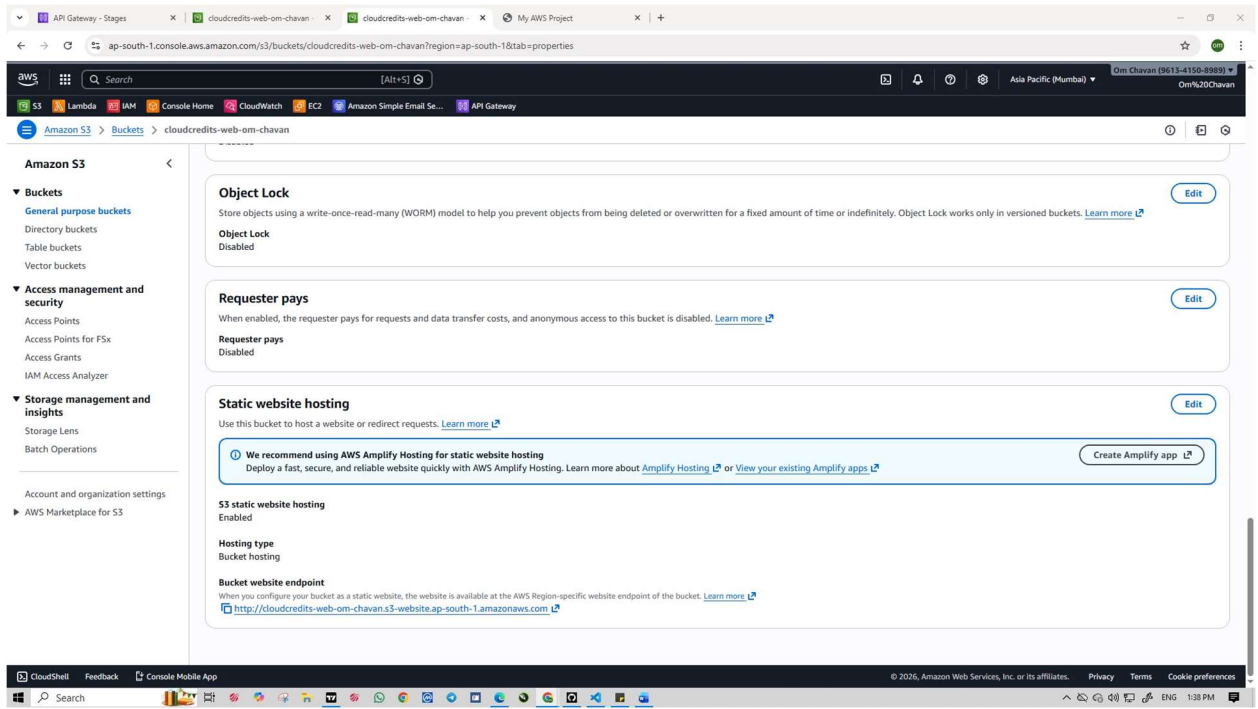
Step 1: Update your index.html file by pasting the **Invoke URL** into your fetch script.

Step 2: Re-upload the updated index.html to your S3 bucket.

Step 3: Open your **S3 Bucket Website Endpoint** in a browser and click your website button.

Step 4: Verify that the message from your Lambda function appears on the screen.





Cloud AI with AWS - Project 4

[Fetch Backend Message](#)

